



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

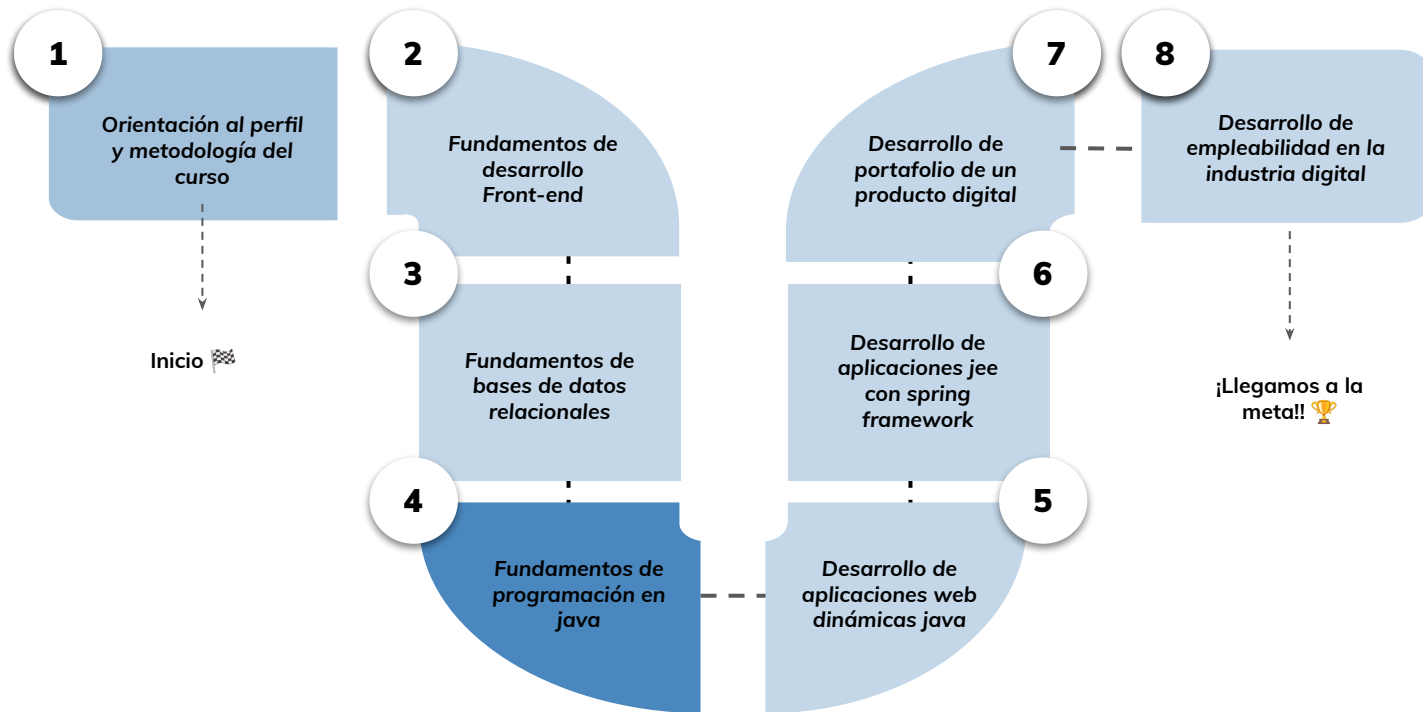


› El entorno java para la programación algoritmos- Parte 2

Aprendizaje Esperado 3: Utilizar la sintaxis básica del lenguaje java para la construcción de programas que resuelven un problema de baja complejidad.

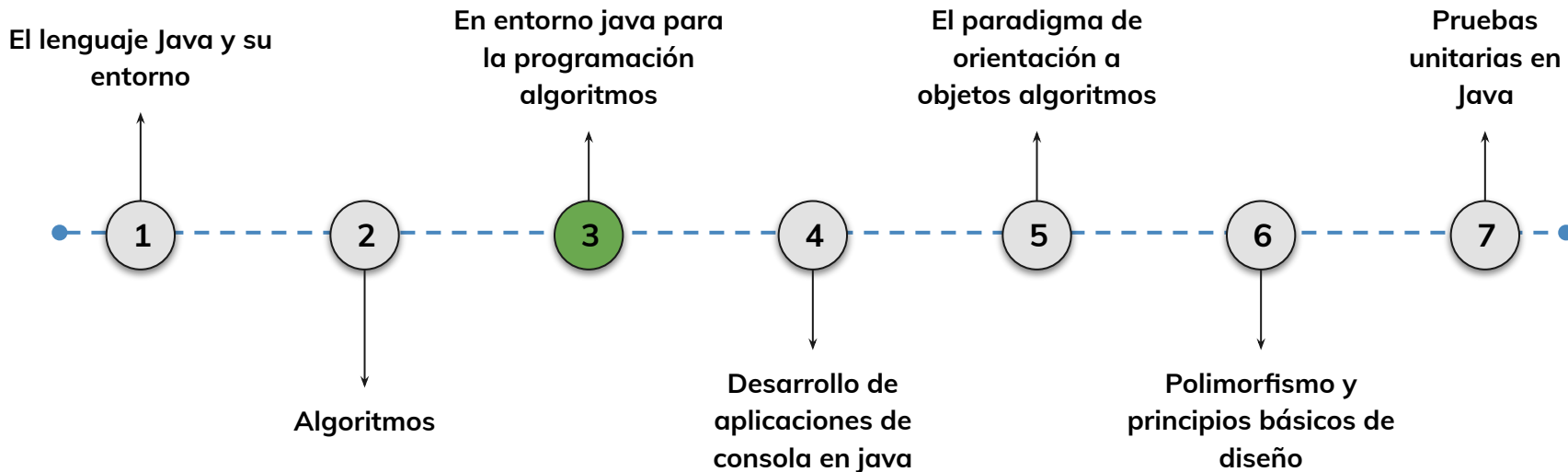
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



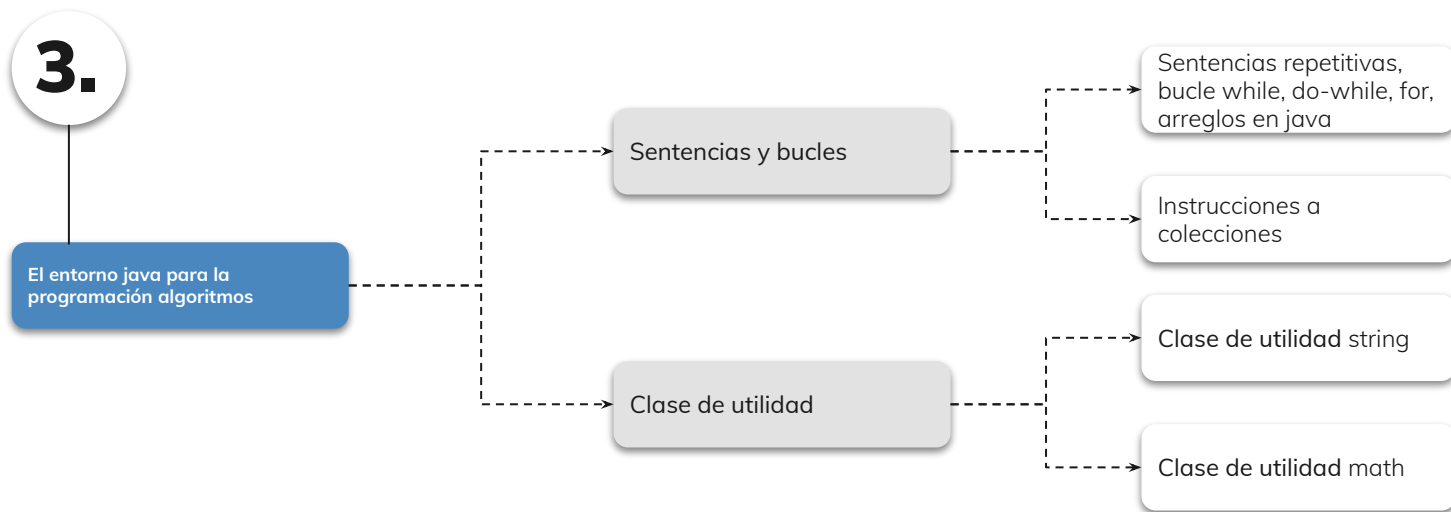
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Implementar sentencias repetitivas para controlar el flujo de un programa Java.
- Conocer y manipular arreglos en Java
- Comprender el concepto de las colecciones
- Conocer y manipular las funciones asociadas a la clase String
- Conocer y manipular las funciones asociadas a la clase Math

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Reconocimos la sintaxis e implementación de las variables en Java.
- Comprendimos los tipos de datos y su manipulación.
- Conocimos e implementamos el uso de operadores en programas Java.
- Reconocimos la estructura de las expresiones.
- Reconocimos las sentencias condicionales

➤ Sentencias Repetitivas

< > Sentencias repetitivas

¿Qué son las sentencias repetitivas?:

Las estructuras que repiten una secuencia de instrucciones un número determinado de veces se denominan **bucles**, y se denomina iteración al hecho de repetir la ejecución de una secuencia de acciones.

Todo bucle se encuentra asociado a una condición, que es la que va a determinar cuándo y cuántas veces se repite.



➤ Bucle while

< > While

Sintaxis en Java

Esta estructura de control **repite un conjunto de instrucciones mientras una condición se cumpla (true)** , en cuanto la condición no se cumple el bucle deja de ejecutarse. Es por ello que dentro del bloque deberán existir sentencias que modifiquen la evaluación de la expresión, ya que de no hacerse se podría entrar en un bucle infinito.

```
while ( <condición> ) {  
    <sentencias>;  
}
```

En el caso de que la condición se evalúe por primera vez como false, el ciclo no será ejecutado ni una vez.



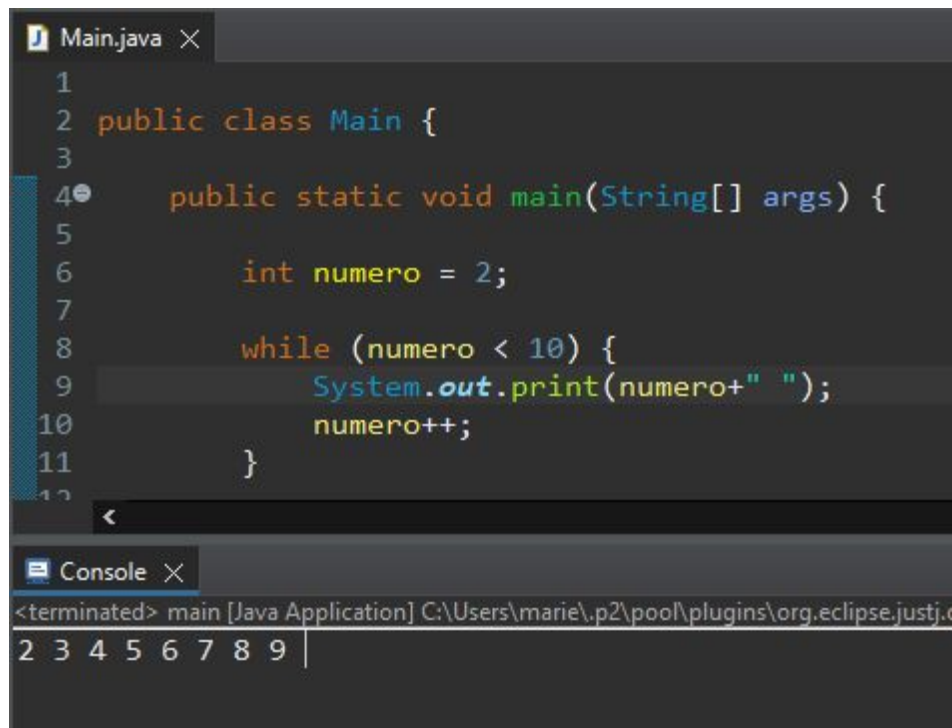
< > While

Iteración:

En este ejemplo utilizamos la variable **número** como **iterador**. Es por esto que es necesario **incrementar** esta variable (línea 10) para que se cumpla en algún momento la **condición de finalización** del bucle.

Podemos ver en la salida por consola como se imprimen todos los números menores a diez.

Truco: para escribir en una misma línea, se utiliza la función **print** (sin “\n”) del System.out.



The screenshot shows an IDE with two panels. The top panel, titled 'Main.java', contains the following Java code:

```
1
2 public class Main {
3
4     public static void main(String[] args) {
5
6         int numero = 2;
7
8         while (numero < 10) {
9             System.out.print(numero+" ");
10            numero++;
11        }
12    }
13}
```

The bottom panel, titled 'Console', shows the output of the program:

```
<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.c
2 3 4 5 6 7 8 9 |
```

LIVE CODING

Ejemplo en vivo

while:

Vamos a ver un ejemplo de bucle while dónde la variable iterador será modificada por el usuario.

1. Desarrollar un bucle while que lea números ingresados por el usuario, hasta que se ingrese 0 (cero).

Tiempo: 15 minutos

➤ Bucle do-while

< > Do-While

Sintaxis en Java

En el caso de la estructura repetitiva do-while el funcionamiento es casi el mismo que el de while. Pero con una diferencia, **primero se ejecuta el bloque de sentencias y luego se evalúa la expresión** . Al igual que en el while, en el bloque de sentencias deberemos modificar la condición para poder salir del bucle.

```
do{  
    <sentencias>;  
}while ( <condición>);
```

Por lo tanto, **siempre se ejecutará al menos una vez.**

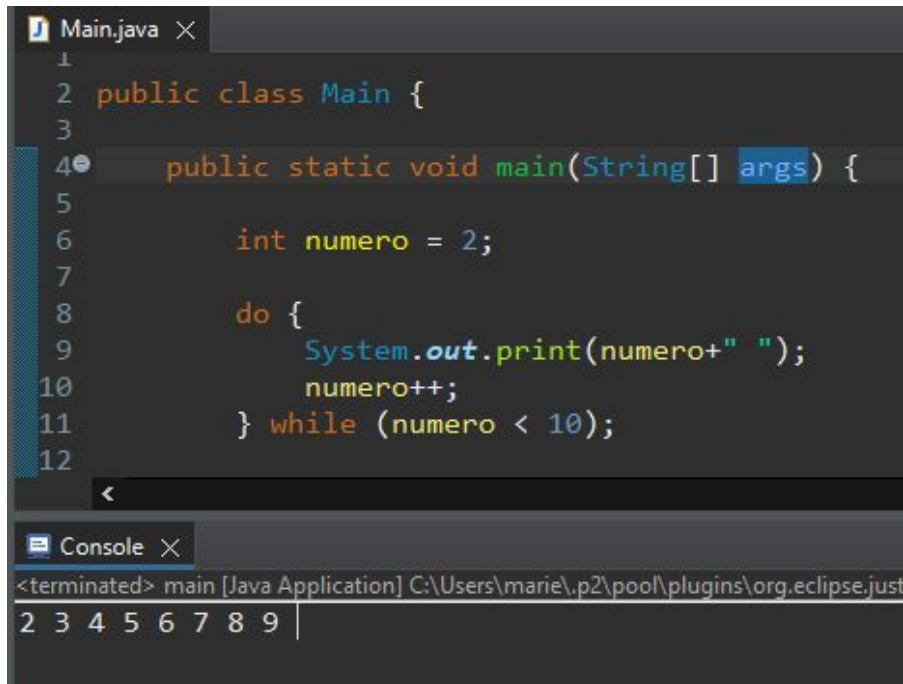


< > Do-while

Iteración:

En este ejemplo utilizamos nuevamente la variable **número** como **iterador**. Es por esto que es necesario **incrementar** esta variable (línea 10) para que se cumpla en algún momento la **condición de finalización** del bucle.

Podemos ver en la salida por consola como se imprimen todos los números menores a diez.



```
1
2 public class Main {
3
4     public static void main(String[] args) {
5
6         int numero = 2;
7
8         do {
9             System.out.print(numero+" ");
10            numero++;
11        } while (numero < 10);
12
13    }
14 }
```

Console

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.just

2 3 4 5 6 7 8 9 |

LIVE CODING

Ejemplo en vivo

do - while:

Vamos a ver un ejemplo de bucle do-while dónde la variable iterador será modificada por una operación aritmética.

1. Desarrollar un bucle do-while en el cual se ingrese un valor límite positivo, y a continuación solicite números al usuario hasta que la suma de los números introducidos supere el límite inicial.

Tiempo: 15 minutos



Ejercicio N° 1

< > Iterando

La importancia de los bucles: 🙌

Los bucles son estructuras fundamentales en programación. Permiten repetir un bloque de código de forma controlada. De esta manera se puede ejecutar tareas iterativas de manera simple y con pocas líneas de código. **Las estructuras repetitivas son herramientas de uso constante en programación.** Practiquemos un poco...

Consigna: ✍️

Escribir un programa que sume los números del 1 al 10 utilizando.

- 1- bucle while.
- 2- bucle do-while
- 3- bucle for

Tiempo 🕒: 20 minutos

Paso a paso: ⚙️

Recuerda definir los iteradores en los bucles correspondientes. Dejamos un ejemplo de bucle while:

```
while(i <= 10) {  
    suma = suma + i;  
    i++;  
}
```



➤ Bucle for

< > For

Sintaxis en Java

La estructura for proporciona una forma compacta de recorrer un rango de valores cuando la cantidad de veces que se debe iterar un bloque de código es conocida.

```
for (<inicialización>; <terminación>; <incremento>) {  
    <sentencias>;  
}
```

La expresión **<inicialización>** inicializa el bucle y se ejecuta una sola vez al comienzo de la ejecución del bucle. El bucle termina cuando al evaluar la expresión **<terminación>** el resultado que se obtiene es **false**.

La expresión **<incremento>** se invoca después de cada iteración que realiza el bucle; esta expresión incrementa o decrementa un valor **hasta que se cumpla la condición de <terminación>** del bucle.

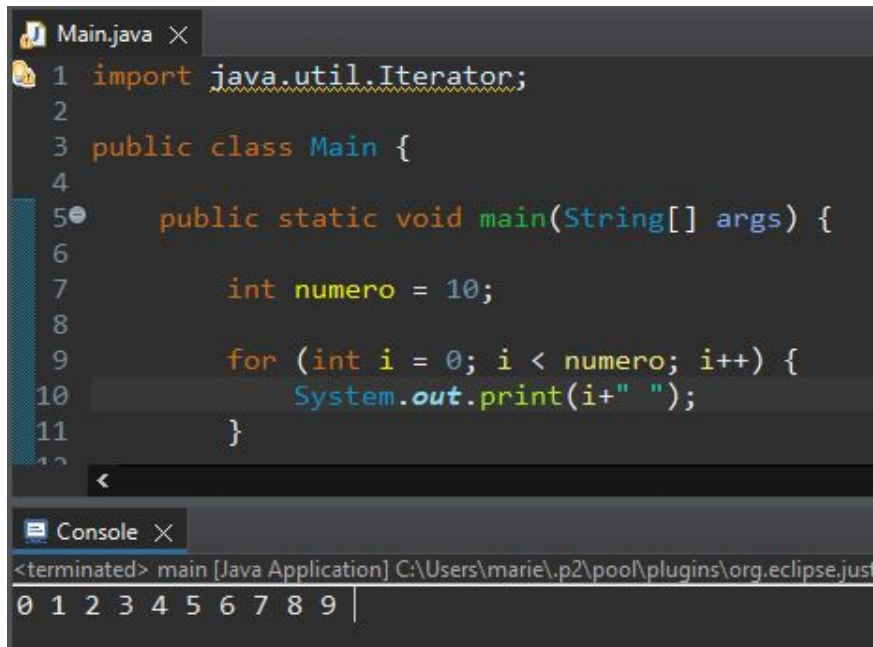


< > For

Iteración:

En este ejemplo podemos ver que **la variable de incremento ya está definida por el bucle** (línea 9, `i++`), por lo tanto usamos el iterador `i` para imprimir los números mientras se cumpla la condición.

El primer número que se imprime es el **cero**, porque es el número de **inicialización de i** (línea 9, `i = 0`).



```
Main.java x
1 import java.util.Iterator;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         int numero = 10;
8
9         for (int i = 0; i < numero; i++) {
10             System.out.print(i+" ");
11         }
12     }
13 }
```

Console x

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.just

0 1 2 3 4 5 6 7 8 9 |



LIVE CODING

Ejemplo en vivo

for:

Vamos a ver un ejemplo de bucle for dónde la variable iterador ya se encuentra previamente definida. El bucle debe dar 10 vueltas.

1. Desarrollar un bucle for en el cual se muestre por pantalla la tabla del 5. Se debe mostrar hasta 5x20.

Tiempo: 15 minutos

< > Bucles

¿Cuál usar?:

Como regla general podemos afirmar que se utilizará el bucle **for** cuando se conozca de antemano **el número exacto de veces que debe repetirse** un determinado bloque de instrucciones.

Se utilizará el bucle **do-while** cuando no se conoce exactamente el número de veces que se ejecutará el bucle, pero **se sabe que por lo menos se ha de ejecutar una vez** .

Se utilizará el bucle **while** cuando es posible **que no deba ejecutarse ninguna vez**.



➤ Arreglos en Java

< > Arreglos

¿Qué es un arreglo?:

Un arreglo, también conocido como **array**, es una estructura de datos que puede **almacenar una colección de elementos del mismo tipo**. En Java, los arrays tienen un **tamaño fijo** que se define en el momento de su creación. Cada elemento del array se puede acceder mediante un **índice numérico** que representa su posición en el array.

Para declarar un array en Java, debes especificar su tipo y su tamaño:

```
int[] numeros = new int[4];
```

Los corchetes indican que esta variable es del tipo arreglo y el prefijo int informa que almacenará números enteros. Además, especificamos su tamaño en 4 espacios.



< > Arreglos

Índices de un arreglo:

```
int[] numeros = new int[4];
```

Este array está vacío, es como una variable normal, pero con cuatro espacios. Para rellenarlos, hay que asignar datos a cada parte del contenedor.

Para asignar valores, tienes que utilizar el **índice** de los arrays. Se cuenta **desde la posición 0** hasta la que tenga el array. En este caso, tiene cuatro espacios y, por lo tanto, el índice va del 0 al 3 (4 valores en total).

```
int[] numeros = new int[4];  
numeros[0] = 10;  
numeros[1] = 20;  
numeros[2] = 30;  
numeros[3] = 40;
```

< > Arreglos

Acceder a una posición del arreglo:

Después de almacenar valores, llega el momento de utilizarlos. Para usar uno de estos cuatro valores, lo haremos llamando al array y la posición que queramos:

```
7      int[] numeros = new int[4];
8      numeros[0] = 10;
9      numeros[1] = 20;
10     numeros[2] = 30;
11     numeros[3] = 40;
12
13     System.out.print("El valor de la primera posición del array es: " + numeros[0] + ".");
14
```

<

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_17.0.7.v20230425-1502\jre\bin\javaw.exe (15 a

El valor de la primera posición del array es: 10.

< > Arreglos

Declarar e inicializar array en Java:

También puedes inicializar un array con valores específicos en el momento de su creación. Por ejemplo:

```
int[] numeros = {10, 20, 30, 40};
```

De esta forma, no hay que indicar la longitud, ya que los propios elementos la especifican. **Si hay cuatro elementos, el array toma una longitud de índice de 4** .



< > Arreglos

Acceder con bucle for a todos los elementos de un arreglo:

Se puede automatizar el acceso a todas las partes de un array con un bucle. En esta ocasión, vamos a utilizar el bucle for, que es el más usado para recorrer arreglos.

Truco: la función **.length** devuelve el tamaño del arreglo.

```
7      int[] numeros = {10, 20, 30, 40};
8
9      for (int i = 0; i < numeros.length; i++) {
10         System.out.println("El valor es: " + numeros[i] + ".");
11     }
12
13
14
```

<

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64

El valor es: 10.
El valor es: 20.
El valor es: 30.
El valor es: 40.

< > Arreglos

El bucle for-each de Java

Desde hace bastante tiempo se añadió una herramienta muy especial para este tipo de tareas tan frecuentes. Se trata del bucle **for-each o enhanced loop**.

El resultado es el mismo, y Java lo entiende correctamente. Solo hay que darle un **nombre al iterador, dos puntos y el nombre del array que se quiere iterar** de arriba a abajo. No hay que usar ni length ni incrementos. Es más cómodo, limpio y fácil de escribir.

```
7      int[] numeros = {10, 20, 30, 40};
8
9      for (int i : numeros) {
10         System.out.println("El valor es: " + i + ".");
11     }
12
13
14
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.

El valor es: 10.
El valor es: 20.
El valor es: 30.
El valor es: 40.

< > Arreglos bidimensionales

Matrices:

Los arreglos bidimensionales o matrices tienen el mismo tratamiento que los arrays. La diferencia de sintaxis se centra en la creación:

Declaración y creación de una Matriz

```
int[][] arregloM = new int [Filas][Columnas] ;
```

De esta manera podemos crear dos dimensiones diferentes para el mismo arreglo, es por esto que lleva dos corchetes.



<> Arreglos

Acceder con dos bucle for a todos los elementos de una matriz:

Para recorrer una matriz, es necesario que ingresemos tanto a filas como a columnas. Para esto vamos a utilizar un bucle for anidado por cada dimensión de la matriz.

```
7      int[][] matriz = new int [2][2];
8
9      matriz[0][0] = 10;
10     matriz[0][1] = 20;
11     matriz[1][0] = 30;
12     matriz[1][1] = 40;
13
14
15     for (int i = 0; i < matriz.length; i++) {
16         for (int j = 0; j < matriz.length; j++) {
17             System.out.println("fila "+ i + " columna "+j + " tiene el valor: "+ matriz[i][j]);
18         }
19     }
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.17.0.7.v20230425-1502\jre\bin\javaw.exe (15 ago. 2023)

```
fila 0 columna 0 tiene el valor: 10
fila 0 columna 1 tiene el valor: 20
fila 1 columna 0 tiene el valor: 30
fila 1 columna 1 tiene el valor: 40
```

➤ Introducción a Colecciones

< > Colecciones

¿Qué son las colecciones?:

Una colección **representa un grupo de objetos** , conocidos como elementos. Podemos crear una colección con cualquier tipo de objeto, incluso los creados por nosotros. La única restricción es que **no podremos crear una colección sólo de tipo primitivo** , para eso usaremos los tipos de objetos equivalentes a los primitivos. Por ejemplo: **en vez de int, hay que utilizar Integer** . A continuación veremos los objetos equivalentes a los primitivos de Java.

Tipos de datos	
Primitivos	Objetos
int	Integer
double	Double
long	Long
char	Character
boolean	Boolean
String ya es un objeto, por lo que no tiene tipo primitivo	



Arreglos

Arreglos:

Tamaño fijo : Los arreglos tienen un tamaño fijo que se define al crearlos.

Tipo de datos único : Los arreglos almacenan elementos de un único tipo de dato.

Acceso por índice: En los arreglos se accede a elementos por su índice numérico.

Almacenamiento continuo: Los arreglos son continuos en memoria,

Colecciones

Colecciones:

Tamaño variable : Las colecciones se expanden conforme se agregan elementos.

Múltiples tipos : Las colecciones pueden almacenar objetos de diferentes tipos.

Acceso no por índice : En las colecciones se accede por iteración o claves.

Almacenamiento no continuo: Las colecciones pueden fragmentarse.

LIVE CODING

Ejemplo en vivo

Recorriendo Arrays:

Vamos a ver un ejemplo de manipulación de cadenas en arreglos. Esto es una práctica útil para guardar datos como por ejemplo contraseñas

1. Declarar y recorrer un arreglo que almacene el nombre de 5 países ingresados por el usuario. Mostrar el contenido del arreglo por pantalla.

Tiempo: 20 minutos



Ejercicio N° 2

< > Arreglando

Pongamos en práctica: 🙌

Los arreglos son estructuras que nos permiten almacenar varios datos con un tipo en común. Esto sirve para gestionar una lista de supermercado o para organizar a los jugadores titulares de tu equipo favorito. Son ampliamente utilizados y es por esto que es necesario ponerlo en práctica.

Consigna: 📝

Crear un arreglo para almacenar 3 números reales ingresados por el usuario y mostrar el mayor elemento.

Tiempo 🕒: 25 minutos

Paso a paso: ⚙️

1. Declara el arreglo con tamaño 3
2. Recorrer el arreglo con un bucle for
3. Pedir el numero al usuario dentro del bucle
4. Asignar el valor a cada elemento utilizando el iterador i
5. Luego de almacenar, recorre el arreglo con otro bucle for dónde validarás cuál es el mayor (condicionales)



➤ Clase de utilidad: String

< > Clase String

¿Qué es la Clase String?:

La clase String en Java es una herramienta poderosa **para manipular cadenas de texto** . Con ella puedes concatenar cadenas para unir las, obtener la longitud de una cadena, buscar subcadenas, reemplazar partes de una cadena y mucho más. Además, la clase String **proporciona métodos** para comparar cadenas y realizar operaciones de formato.

La Clase String tiene muchos métodos para manipular cadenas, en ésta clase veremos algunos de ellos.



< > String: concatenar

Concatenación de Strings:

La concatenación de Strings en Java se puede realizar con el método **concat()** que pertenece a la propia clase String.

Con **'texto1.concat(" ")'** le indicamos que queremos concatenar el espacio " " al valor del string de **'texto1'**. Luego, **con un punto, podemos ir añadiendo más concat()**. En este caso, **se añade el valor de 'texto2'**.

```
7      String texto1 = "Hola";
8      String texto2 = "Mundo";
9      String texto3 = texto1.concat(" ").concat(texto2);
10
11      System.out.print(texto3);|
```

Console X

```
<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.  
Hola Mundo
```

< > String: comparar

Comparación de Strings:

La comparación de Strings en Java se puede realizar con el método **equals()** que pertenece a la propia clase String.

Con '**texto1.equals(texto2)**' le indicamos que queremos comparar si ambas cadenas son iguales. Esta función nos devuelve un valor booleano.

```
7      String texto1 = "Hola";
8      String texto2 = "Mundo";
9      Boolean iguales = texto1.equals(texto2);
10
11      System.out.print(iguales);
12  }
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openj
false

< > String: índices

Concatenación de Strings:

Los Strings están formados por un conjunto variable de caracteres. Entonces, para poder manipular individualmente cada uno de estos, existe lo que se conoce como índice. El índice de los Strings **empieza por el valor entero 0** y va aumentando en 1 con cada caracter.

El método indexOf() devuelve la posición de la primera ocurrencia de una subcadena en una cadena.

```
4
5 public static void main(String[] args) {
6
7     String texto1 = "Hola Mundo";
8     int valorIndice = texto1.indexOf("Mundo");
9
10    System.out.print(valorIndice);
11 }
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk

5

< > String: reemplazar

El método replace()

El método replace() reemplaza todas las ocurrencias de una subcadena con otra subcadena. Se esperan dos argumentos en la llamada. Sencillamente, se busca el primer valor y se reemplaza en todas las ocurrencias.

```
6
7     String texto1 = "hola, este String tiene más de un hola. Está hecho para probar "
8         + "el metodo replace con la palabra hola.";
9
10    String nuevoTexto = texto1.replace("hola", "adiós");
11
12    System.out.print(nuevoTexto);
13
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.7.v20230425-1502\jre\bin\javaw.exe (16 ago. 2023)

adiós, este String tiene más de un adiós. Está hecho para probar el metodo replace con la palabra adiós.



< > String: subcadena

El método substring()

El método substring() devuelve una subcadena que **empieza desde la posición indicada** como argumento.

```
7      String texto1 = "Hola Mundo";
8      String character = texto1.substring(2);
9
10     System.out.print(character);
11 }
12
13 }
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp

la Mundo

LIVE CODING

Ejemplo en vivo

Manipulando Strings:

Vamos a ver el funcionamiento de algunas funciones de Strings que pueden ser muy útiles a la hora de desarrollar nuestros programas.

1. Realizar un programa que solo permita introducir frases o palabras de **8 de longitud**. Si el usuario ingresa una cadena de 8 de longitud se deberá imprimir un mensaje por pantalla que diga "CORRECTO" e **imprimir la frase en mayúsculas**, en caso contrario, se deberá imprimir "INCORRECTO" y toda **la frase en minúsculas**.

Tiempo: 25 minutos

➤ Clase de utilidad: Math

< > Clase Math

¿Qué es la clase Math?

Con la clase Math puedes realizar **operaciones aritméticas** básicas, como sumar, restar, multiplicar y dividir números. También puedes utilizarla para realizar cálculos más complejos, como redondear números, calcular raíces cuadradas, generar números aleatorios y mucho más.

Además de las operaciones matemáticas, la clase Math también contiene **constantes** matemáticas útiles, como **PI** (π) y **E** (número de Euler).

Para invocar los siguientes métodos, simplemente debes colocar la palabra **Math.** y a continuación el nombre del método que quieres utilizar. Veamos algunos de los métodos más utilizados.



< > **Math:** redondeo y truncamiento

Métodos ceil(), floor() y round():

La Clase Math en Java proporciona varios métodos para redondear y truncar números decimales. Estos métodos son útiles cuando necesitamos ajustar los valores numéricos a ciertos límites o para realizar cálculos que requieren valores enteros.

- **Math.ceil():** Redondea hacia arriba, es decir, devuelve el entero más pequeño que es mayor o igual al número dado.
- **Math.floor():** Redondea hacia abajo, es decir, devuelve el entero más grande que es menor o igual al número dado.
- **Math.round():** Redondea el valor de un número real al entero más cercano.

Veamos el comportamiento en código:



< > Math.ceil()

```
3      double numero = 5.3;
4      double resultado = Math.ceil(numero);
5      System.out.println("Redondeando hacia arriba el número "
6                          + numero + ": " + resultado);
```

<

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32

Redondeando hacia arriba el número 5.3: 6.0



< > Math.round()

```
3     double numero = 5.5;  
4     double resultado = Math.round(numero);  
5     System.out.println("Redondeando al entero más cercano el número "  
6                          + numero + ": " + resultado);
```

Console X

```
<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.7.v  
Redondeando al entero más cercano el número 5.5: 6.0
```



< > Math: potenciación y radicación

Métodos `pow()`, `sqrt()` y `cbrt()`:

La Clase Math también proporciona funciones para realizar operaciones de potenciación y radicación. Estos métodos son útiles cuando necesitamos elevar números a potencias específicas o calcular raíces de diferentes órdenes.

- **Math.pow():** Eleva un número a una potencia determinada.
- **Math.sqrt():** Calcula la raíz cuadrada de un número.
- **Math.cbrt():** Calcula la raíz cúbica de un número.

Veamos el comportamiento en código:



< > Math.pow()

```
3      double base = 5;  
4      double exponente = 3;  
5      double resultado = Math.pow(base, exponente);  
6      System.out.println("El número " + base +  
7                          " elevado a la potencia " + exponente +  
8                          " es: " + resultado);
```

Console ×

```
<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64  
El número 5.0 elevado a la potencia 3.0 es: 125.0
```



< > Math.sqrt()

```
3     double numero = 25;  
4     double resultado = Math.sqrt(numero);  
5     System.out.println("La raíz cuadrada del número " + numero + " es: " + resultado);
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.7.v20230425-1502\jre\bin\javaw.exe

La raíz cuadrada del número 25.0 es: 5.0



< > Math.cbrt()

```
3      double numero = 27;  
4      double resultado = Math.cbrt(numero);  
5      System.out.println("La raíz cúbica del número " + numero + " es: " + resultado);|
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.7.v20230425-1502\jre\bin\javaw.exe
La raíz cúbica del número 27.0 es: 3.0

< > Math: generación de números aleatorios

Método `random()`:

La Clase Math en Java incluye métodos para generar números aleatorios. Estos números pueden ser útiles en una variedad de aplicaciones, como juegos, simulaciones y algoritmos de optimización.

- **Math.random():** Genera un número aleatorio de tipo double entre 0 (inclusive) y 1 (exclusivo).

Veamos el comportamiento en código:



< > Math.random()

```
3      double aleatorio = Math.random();
4      System.out.println("Número aleatorio entre 0 y 1: " + aleatorio);
5
6  <
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.7
Número aleatorio entre 0 y 1: 0.99407537320504



< > Math.random()

Para generar un número entero entre 0 y 9, Hay que escribir la siguiente sentencia:

```
int aleatorio = (int) (Math.random () * 10);
```

De esta manera al multiplicar el valor por 10, el rango de valores posibles se convierte en: 0.0
≤ numero < 10.0

```
3
4      int aleatorio = (int) (Math.random () * 10);
5
6      System.out.println("Número aleatorio entre 0 y 9: " + aleatorio);|
7
<
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.7.v20230425-

Número aleatorio entre 0 y 9: 6



< > Math: comparación y selección

Métodoa `min()`, `max()` y `abs()`:

La Clase Math en Java también proporciona funciones para comparar y seleccionar valores numéricos. Estos métodos son útiles para determinar el mínimo, el máximo o el valor absoluto de los números, así como para realizar otras operaciones de comparación y selección.

- **Math.min()** : Retorna el menor de dos valores.
- **Math.max()** : Retorna el mayor de dos valores.
- **Math.abs()** : Retorna el valor absoluto de un número.

Veamos su aplicación en código:



< > Math.min()

```
4      int num1 = 5;
5      int num2 = 10;
6      int minimo = Math.min(num1, num2);
7      System.out.println("El menor entre " + num1 + " y "
8                          + num2 + " es: " + minimo);
9
```

<

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win
El menor entre 5 y 10 es: 5



< > Math.max()

```
4      int num1 = 5;
5      int num2 = 10;
6      int maximo = Math.max(num1, num2);
7      System.out.println("El mayor entre " + num1 + " y "
8                          + num2 + " es: " + maximo);
9
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w

El mayor entre 5 y 10 es: 10

< > Math.abs()

```
4      int num = -10;  
5      int absoluto = Math.abs(num);  
6      System.out.println("El valor absoluto de " + num + " es: " + absoluto);  
7
```

Console X

<terminated> main [Java Application] C:\Users\marie\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.7.v20230425-1502
El valor absoluto de -10 es: 10



LIVE CODING

Ejemplo en vivo

Math.¿qué?:

Vamos a poner en práctica lo aprendido resolviendo en vivo una operación aritmética bastante frecuente:

1. Desarrollar un programa que tome como dato de entrada un número que corresponde a la longitud del radio una esfera y que calcule y escriba el volumen de la esfera que se corresponde con dicho radio.

La fórmula para calcular el volumen de la esfera es $v = (4/3) * \pi * r^3$

Tiempo: 25 minutos



Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Aprender a aplicar bucles de repetición**
- ✓ **Comprender la sintaxis e implementación de los arreglos en Java**
- ✓ **Comprender la importancia e implementación de la clase de utilidad `String` y sus métodos asociados.**
- ✓ **Comprender la importancia e implementación de la clase de utilidad `Math` y sus métodos asociados.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

